

✓ Install Dependencies (Google Colab)

```
!pip install --upgrade openai sentence-transformers spacy nltk requests gradio pandas \
    langchain-community langchain-openai langgraph \
    pymupdf chromadb
```

 [Show hidden output](#)

✓ Download nltk

```
!python -m spacy download en_core_web_sm
```

 [Show hidden output](#)

✓ Import Required Libraries

```
import os
import re
import csv
import json
import time
import nltk
import fitz
import spacy
import shutil
import uuid
import torch
import tiktoken
import chromadb
import openai
import requests
import subprocess
import datetime
import pathlib
import threading
import logging
import gradio as gr
import pandas as pd
import plotly.express as px
from openai import OpenAI
from functools import partial
from openai import AsyncOpenAI
from collections import Counter
from langchain.agents import Tool
import plotly.graph_objects as go
from google.colab import userdata
import xml.etree.ElementTree as ET
from chromadb.config import Settings
from chromadb.utils import embedding_functions
from datetime import timedelta, timezone
from unittest import TestLoader, TextTestRunner
from langchain.schema import HumanMessage
from langchain_community.chat_models import ChatOpenAI
from openai import RateLimitError, BadRequestError
from chromadb.utils import embedding_functions
from langgraph.prebuilt import create_react_agent
from langchain.agents import initialize_agent, Tool
from typing import List, Dict, Any, Tuple, Union, Optional
from sentence_transformers import SentenceTransformer, util
from concurrent.futures import ThreadPoolExecutor, as_completed
```

```
import nltk
nltk.download('wordnet')
```

 [Show hidden output](#)

✖ Retrieve API Keys (OpenAI)

```
openai_api_key = userdata.get("OPENAIKEY")
```

✖ Set OpenAI API Key

```
def set_openai_api_key(api_key):
    if api_key:
        os.environ["OPENAI_API_KEY"] = api_key
        print("✅ OpenAI API key set successfully.")
    else:
        print("⚠️ OpenAI API key is missing.")

set_openai_api_key(openai_api_key)

# Initialize OpenAI Client
client = OpenAI() # Now works with proper import

#async client
aclient = AsyncOpenAI(api_key=openai_api_key)

# Set up basic logging
logging.basicConfig(level=logging.INFO, format="%(levelname)s: %(message)s")
```

 [Show hidden output](#)

✖ TIM Pathway Mapper

```
class TIMPathwayMapper:
    PATHWAY_PROFILES = {
        "MDTE": {
            "population": "digital transformation experts, tech leaders",
            "weights": {"semantic": 0.5, "engagement": 0.4, "recency": 0.1},
            "concepts": ["AI adoption", "blockchain", "digital ecosystems", "IoT"],
            "context": "technology-driven organizations, cross-border solutions"
        },
        "MEnt": {
            "population": "entrepreneurs, startup founders, venture capitalists",
            "weights": {"semantic": 0.7, "engagement": 0.2, "recency": 0.1},
            "concepts": ["business scaling", "startup development", "venture funding"],
            "context": "new venture creation, innovative ecosystems"
        },
        "MABA": {
            "population": "business analysts, data scientists, tech professionals",
            "weights": {"semantic": 0.6, "engagement": 0.3, "recency": 0.1},
            "concepts": ["predictive analytics", "big data", "AI-based decision-making"],
            "context": "data-driven business environments"
        },
        "MEng": {
            "population": "engineering researchers, R&D specialists",
            "weights": {"semantic": 0.4, "engagement": 0.4, "recency": 0.2},
            "concepts": ["prototyping", "tech transfer", "product development"],
            "context": "applied engineering, industry-university collaboration"
        }
    }

    def __init__(self):
        self.model = SentenceTransformer('all-mpnet-base-v2')
        self.download_thresholds = {
            "MDTE": 500, # Base threshold for tech papers
            "MEnt": 300, # Startups/entrepreneurship papers
            "MABA": 400, # Business analytics
            "MEng": 250 # Engineering research
        }

    def map_to_pathway(self, pcc_elements: dict) -> str:
        """Enhanced mapping with engagement awareness"""
```

```

best_match = "General"
max_score = 0

context_embedding = self.model.encode(pcc_elements["context"], convert_to_tensor=True)
concept_embedding = self.model.encode(pcc_elements["concept"], convert_to_tensor=True)

for pathway, profile in self.PATHWAY_PROFILES.items():
    profile_context = self.model.encode(profile["context"], convert_to_tensor=True)
    profile_concept = self.model.encode(" ".join(profile["concepts"]), convert_to_tensor=True)

    # Weighted similarity scoring
    context_sim = util.pytorch_cos_sim(context_embedding, profile_context).item()
    concept_sim = util.pytorch_cos_sim(concept_embedding, profile_concept).item()

    score = (context_sim * profile["weights"]["semantic"] +
             concept_sim * profile["weights"]["engagement"] +
             0.1 * profile["weights"]["recency"])

    if score > max_score:
        max_score = score
        best_match = pathway

return best_match if max_score >= 0.5 else "General"

def get_engagement_threshold(self, pathway: str) -> int:
    """Returns minimum download threshold for pathway"""
    return self.download_thresholds.get(pathway, 200)

```

✓ PCC Extractor

```

class PCCExtractor:
    def __init__(self, client: OpenAI): # Direct OpenAI client injection
        self.nlp = spacy.load("en_core_web_sm")
        self.client = client # Use client directly

    def extract_pcc_elements(self, query: str) -> Dict[str, str]:
        """Simplified extraction without augmentation"""
        try:
            response = self.client.chat.completions.create(
                model="gpt-4-1106-preview",
                response_format={"type": "json_object"},
                messages=[{
                    "role": "system",
                    "content": "Extract PCC elements as JSON with population, concept, context as lists"
                }, {
                    "role": "user",
                    "content": query
                }]
            )
            pcc_data = json.loads(response.choices[0].message.content)
            return {
                "population": " ".join(pcc_data.get("population", [])),
                "concept": " ".join(pcc_data.get("concept", [])),
                "context": " ".join(pcc_data.get("context", []))
            }
        except Exception as e:
            print(f"GPT extraction failed: {str(e)}")
            return {
                "population": "general population",
                "concept": "general concept",
                "context": "general context"
            }

    def _validate_field(self, value: Any, field_name: str) -> str:
        """Ensure proper formatting of PCC components"""
        if isinstance(value, list):
            return " ".join([str(item) for item in value])
        if isinstance(value, str):
            return value
        print(f"Invalid format for {field_name}, using default")
        return f"general {field_name}"

```

✓ Review Question Formulation Module

```
class ReviewQuestionFormulator:
    def __init__(self, framework: str = "PCC"):
        """
        Initialize the formulator with a specific framework.
        Default is PCC (Population, Concept, Context).
        """
        self.framework = framework
        # Initialize the OpenAI client
        self.client = OpenAI()

    def extract_keywords(self, research_objective: str) -> List[str]:
        """Fixed keyword extraction method"""
        prompt = f"""Extract 5-10 keywords from this research objective:
        {research_objective}
        Keywords (comma-separated):"""

        response = self.client.chat.completions.create(
            model="gpt-4-1106-preview",
            messages=[{"role": "user", "content": prompt}],
            temperature=0
        )

        output = response.choices[0].message.content.strip()
        return [kw.strip() for kw in output.split(",") if kw.strip()]

    def generate_review_questions(self, problem_statement: str, num_variations: int = 3) -> List[str]:
        """Fixed generator with strict format enforcement"""
        prompt = f"""You are an expert academic researcher. Given this problem statement:
        {problem_statement}

        Generate exactly {num_variations} review questions using this EXACT format for each:

        [N]. Review Question: [question]
        P: [Population]
        C: [Concept]
        C: [Context]

        Rules:
        1. Use numbers 1-{num_variations} only
        2. Never create sub-bullets or nested lists
        3. Maintain 3-space indentation for PCC elements
        4. Never combine multiple questions in one list item
        5. Strictly follow the line break pattern shown

        Example valid output:
        1. Review Question: How does X affect Y in Z?
        P: Adult population
        C: Cardiovascular health
        C: Urban environments

        Problem Statement: {problem_statement}
        Output:"""
```

Generate exactly {num_variations} review questions using this EXACT format for each:

```
[N]. Review Question: [question]
P: [Population]
C: [Concept]
C: [Context]
```

Rules:

1. Use numbers 1-{num_variations} only
2. Never create sub-bullets or nested lists
3. Maintain 3-space indentation for PCC elements
4. Never combine multiple questions in one list item
5. Strictly follow the line break pattern shown

Example valid output:

1. Review Question: How does X affect Y in Z?
 - P: Adult population
 - C: Cardiovascular health
 - C: Urban environments

Problem Statement: {problem_statement}

Output:"""

```
        response = self.client.chat.completions.create(
            model="gpt-4-1106-preview", # More reliable for formatting
            messages=[{"role": "user", "content": prompt}],
            temperature=0
        )

        return self._parse_questions(response.choices[0].message.content)

    def _parse_questions(self, output: str) -> List[str]:
        """Robust parsing of structured output"""
        questions = []
        current_question = []

        for line in output.split('\n'):
            line = line.strip()
            if line.startswith(tuple(f"{i}." for i in range(1, 10))):
                if current_question:
                    questions.append('\n'.join(current_question))
                    current_question = []
                current_question.append(line)
            else:
                current_question.append(line)

        if current_question:
            questions.append('\n'.join(current_question))
```

```

        current_question.append(line)
    elif line.startswith(('P:', 'C:')) and current_question:
        current_question.append(line)

    if current_question:
        questions.append('\n'.join(current_question))

    return questions[:3] # Enforce max 3 questions

def generate_review_questions_with_pcc(self, problem_statement: str, num_variations: int = 3) -> List[Dict[str, Any]]:
    """Fixed generation with direct PCC extraction"""
    questions = self.generate_review_questions(problem_statement, num_variations)
    pcc_extractor = PCCExtractor(client=self.client) # Pass formulator's client

    return [{
        "question": q,
        "pcc": pcc_extractor.extract_pcc_elements(q)
    } for q in questions]

```

✓ Query Augmentation Module

```

class QueryAugmenter:
    def __init__(self, openai_client: OpenAI, tim_mapper: "TIMPathwayMapper"):
        self.client = openai_client
        self.tim_mapper = tim_mapper
        self.max_retries = 3
        self.retry_delay = 2

    def augment_query_with_paraphrasing(self, query: str, pcc: dict, num_variations: int = 3) -> List[str]:
        try:
            response = self.client.chat.completions.create(
                model="gpt-3.5-turbo",
                messages=[{
                    "role": "system",
                    "content": f"Generate {num_variations} academic query variations including: {pcc.get('concept')} concepts"
                }, {
                    "role": "user",
                    "content": f"Paraphrase this query {num_variations} ways: {query}"
                }]
            )
            return [line.strip() for line in response.choices[0].message.content.split('\n') if line.strip()]
        except Exception as e:
            print(f"Augmentation failed: {str(e)}")
            return []

```

✓ Article Retrieval Module

```

class RateLimiter:
    def __init__(self, requests_per_minute: int = 100):
        self.requests_per_minute = requests_per_minute
        self.last_request_time = 0

    def wait_if_needed(self) -> None:
        current_time = time.time()
        elapsed = current_time - self.last_request_time
        wait_time = max(0, (60 / self.requests_per_minute) - elapsed)
        if wait_time > 0:
            time.sleep(wait_time)
        self.last_request_time = time.time()

class PaperRetriever:
    def __init__(self, api_key: str, tim_mapper: "TIMPathwayMapper"):
        self.tim_mapper = tim_mapper
        self.rate_limiter = RateLimiter(requests_per_minute=60)
        self.max_retries = 3
        self.retry_delay = 5

    def contextualize_query(self, base_query: str, pcc_elements: Dict[str, str]) -> str:
        pathway = self.tim_mapper.map_to_pathway(pcc_elements)

```

```

logging.info(f"Mapped query to pathway: {pathway}")
if pathway in self.tim_mapper.PATHWAY_PROFILES:
    profile = self.tim_mapper.PATHWAY_PROFILES[pathway]
    context_keywords = " ".join(profile["concepts"])
    final_query = f"{base_query} {context_keywords} {profile['context']}"
else:
    final_query = base_query
logging.info(f"Final query used for retrieval: {final_query}")
return final_query

def fetch_from_arxiv(self, query: str, limit: int = 10) -> List[Dict[str, Union[str, int]]]:
    """
    Queries the arXiv API using the provided query and returns a list of paper dictionaries.
    Each paper dictionary includes fields such as title, abstract, published_date, citation_count,
    link, source, authors, and id.
    """
    base_url = "http://export.arxiv.org/api/query"
    params = {
        "search_query": query,
        "start": 0,
        "max_results": limit
    }
    for attempt in range(self.max_retries):
        try:
            self.rate_limiter.wait_if_needed()
            response = requests.get(base_url, params=params, timeout=30)
            response.raise_for_status()
            break
        except requests.exceptions.RequestException as e:
            if attempt < self.max_retries - 1:
                time.sleep(self.retry_delay * (attempt + 1))
            else:
                raise Exception(f"arXiv API request failed: {str(e)}")

    # Parse the XML response from arXiv.
    root = ET.fromstring(response.text)
    ns = {"atom": "http://www.w3.org/2005/Atom", "arxiv": "http://arxiv.org/schemas/atom"}
    papers = []
    for entry in root.findall("atom:entry", ns):
        title_elem = entry.find("atom:title", ns)
        abstract_elem = entry.find("atom:summary", ns)
        published_elem = entry.find("atom:published", ns)
        author_elems = entry.findall("atom:author", ns)
        authors = [author.find("atom:name", ns).text.strip() for author in author_elems if author.find("atom:name", ns) is not None]
        id_elem = entry.find("atom:id", ns)
        article_id = id_elem.text.strip() if id_elem is not None else (
            f"{title_elem.text.strip()}_{published_elem.text.strip()[:4]}"
            if (title_elem is not None and published_elem is not None) else "unknown_id"
        )
        link_elem = None
        for link in entry.findall("atom:link", ns):
            if link.attrib.get("rel") == "alternate":
                link_elem = link
                break
        title = title_elem.text.strip() if title_elem is not None else "No Title"
        abstract = abstract_elem.text.strip() if abstract_elem is not None else "No Abstract"
        published_date = published_elem.text.strip() if published_elem is not None else "Unknown"
        year = published_date.split("-")[0] if published_date != "Unknown" else "Unknown"

        # Remove references to download count. Set default citation_count to 0.
        citation_count = 0

        link = link_elem.attrib.get("href", "No Link Available") if link_elem is not None else "No Link Available"

        # Extract DOI if available (this assumes arXiv provides it under the "arxiv:doi" tag)
        doi_elem = entry.find("arxiv:doi", ns)
        doi = doi_elem.text.strip() if doi_elem is not None else None

        paper = {
            "id": article_id,
            "title": title,
            "abstract": abstract,
            "published_date": published_date,
            "year": year,
            "citation_count": citation_count,
            "link": link,
            "source": "arXiv",

```

```

        "authors": authors,
        "doi": doi # Optionally store the DOI
    }
    papers.append(paper)
return papers

def unified_retrieval(self, questions_with_pcc: List[dict], max_results: int = 100) -> List[dict]:
    """
    Pathway-optimized retrieval that concurrently processes multiple queries.
    """
    all_papers = []
    queries = [
        self.contextualize_query(question["question"], question["pcc"])
        for question in questions_with_pcc
    ]
    # Using ThreadPoolExecutor to batch API calls concurrently
    with ThreadPoolExecutor(max_workers=len(queries)) as executor:
        future_to_query = {
            executor.submit(self.fetch_from_arxiv, query, max_results // len(queries)): query
            for query in queries
        }
        for future in as_completed(future_to_query):
            try:
                papers = future.result()
                all_papers.extend(papers)
            except Exception as e:
                logging.error(f"Error fetching papers for query '{future_to_query[future]}': {e}")
    return all_papers

```

✓ Screening and Duplicate Removal Module

```

class ArticleScreening:
    def __init__(self):
        self.model = SentenceTransformer('all-mpnet-base-v2')
        self.similarity_threshold = 0.85

    def screen_articles(self, papers, start_year=None, relevance_keywords=None):
        """
        Returns:
            screened_papers (list): The articles that pass screening.
            exclusion_details (dict): A dictionary counting how many were excluded for each reason.
        """
        screened_papers = []
        exclusion_details = {
            "Publication Year": 0,
            "Relevance": 0
        }

        for paper in papers:
            # Check publication year
            try:
                year = int(paper.get("year", "0"))
            except ValueError:
                year = 0

            # Reason 1: Publication Year
            if start_year and year < start_year:
                exclusion_details["Publication Year"] += 1
                continue # skip adding to screened_papers

            # Reason 2: Relevance
            if relevance_keywords:
                text = (paper.get("title", "") + " " + paper.get("abstract", "")).lower()
                if not any(kw.lower() in text for kw in relevance_keywords):
                    exclusion_details["Relevance"] += 1
                    continue

            # If it wasn't excluded for any reason, it passes screening
            screened_papers.append(paper)

        return screened_papers, exclusion_details

    def remove_duplicates(self, papers: List[Dict]) -> List[Dict]:
        """
        Remove duplicate articles based on a composite identifier: title and publication year.

```

```

remove duplicate articles based on a composite identifier: title and publication year.
If two articles have the same title and year, compare the abstracts using cosine similarity.
If the similarity exceeds a threshold, treat them as duplicates.
"""
unique = []
for paper in papers:
    title = paper.get("title", "").strip()
    year = paper.get("year", "Unknown")
    composite_id = f"{title} | {year}"
    duplicate_found = False
    for existing in unique:
        existing_title = existing.get("title", "").strip()
        existing_year = existing.get("year", "Unknown")
        existing_id = f"{existing_title} | {existing_year}"
        if composite_id == existing_id:
            # Check abstract similarity
            abstract1 = paper.get("abstract", "")
            abstract2 = existing.get("abstract", "")
            if abstract1 and abstract2:
                emb1 = self.model.encode(abstract1, convert_to_tensor=True)
                emb2 = self.model.encode(abstract2, convert_to_tensor=True)
                sim = float(util.pytorch_cos_sim(emb1, emb2)[0][0])
                if sim >= self.similarity_threshold:
                    duplicate_found = True
                    logging.info(f"Removing duplicate (similarity {sim:.2f}): {composite_id}")
                    break
            else:
                # If one or both abstracts are missing, consider it a duplicate.
                duplicate_found = True
                logging.info(f"Removing duplicate (no abstracts): {composite_id}")
                break
    if not duplicate_found:
        unique.append(paper)
return unique

```

✓ Semantic Similarity Module

```

# semantic_similarity_module.py
class SemanticSimilarity:
    def __init__(self, model_name: str = 'all-mpnet-base-v2'):
        self.model = SentenceTransformer(model_name)

    def add_similarity_scores(self, articles: list, research_objective: str) -> list:
        """
        Adds a new 'semantic_similarity_score' key to each article dict,
        based on the cosine similarity between the article's abstract and the research objective.

        Parameters:
            articles (list): A list of article dictionaries. Each dict should have an 'abstract' key.
            research_objective (str): The research objective text to compare against.

        Returns:
            list: The same list of article dictionaries, but each now includes a
                'semantic_similarity_score' key with a float value.
        """
        # Encode the research objective once
        research_emb = self.model.encode(research_objective, convert_to_tensor=True)

        # Compute similarity for each article
        for article in articles:
            abstract = article.get("abstract", "")
            if abstract:
                article_emb = self.model.encode(abstract, convert_to_tensor=True)
                sim = float(util.pytorch_cos_sim(article_emb, research_emb).item())
            else:
                sim = 0.0
            article["semantic_similarity_score"] = sim

        return articles

```


✓ PDF Downloader

```

class PDFDownloader:
    def __init__(self):
        self.chunk_size = 1500
        self.section_headers = [
            "abstract", "introduction", "methods",
            "results", "discussion", "conclusion"
        ]

    def get_full_text(self, article: dict) -> str:
        """
        Download the PDF from the article's link and extract its full text.
        Only processes articles that passed quality assessment (semantic_similarity_score >= 0.5).
        If the PDF cannot be retrieved or processed, or if the article did not pass quality assessment,
        fall back to using the article's abstract.
        """
        # Only process if article has passed quality assessment.
        # If the article's semantic_similarity_score is below 0.5, return the abstract.
        if float(article.get("semantic_similarity_score", 0)) < 0.5:
            return article.get("abstract", "No Abstract Available")

        pdf_url = article.get("pdf_url", None)
        if not pdf_url:
            return article.get("abstract", "No Abstract Available")

        try:
            response = requests.get(pdf_url, timeout=30)
            response.raise_for_status()
            with open("temp.pdf", "wb") as f:
                f.write(response.content)
            doc = fitz.open("temp.pdf")
            full_text = ""
            for page in doc:
                full_text += page.get_text()
            doc.close()
            os.remove("temp.pdf")
            return full_text if full_text else article.get("abstract", "No Abstract Available")
        except Exception as e:
            logging.error(f"Error extracting PDF text: {str(e)}")
            return article.get("abstract", "No Abstract Available")

    def _extract_arxiv_metadata(self, entry) -> dict:
        """Parse arXiv XML metadata including authors"""
        return {
            "title": entry.find("atom:title", {"atom": ""}).text.strip(),
            "authors": [a.text for a in entry.findall("atom:author/atom:name", {"atom": ""})],
            "year": entry.find("atom:published", {"atom": ""}).text[:4],
            "link": entry.find("atom:link[@rel='alternate']", {"atom": ""}).attrib["href"]
        }

    def _chunk_and_categorize(self, text: str) -> List[dict]:
        """Section-aware chunking with citation anchors"""
        chunks = []
        current_section = "preface"
        buffer = []

        for line in text.split('\n'):
            detected_section = self._detect_section_header(line)
            if detected_section:
                if buffer:
                    chunks.append(self._create_chunk(buffer, current_section))
                    current_section = detected_section
                    buffer = []
                buffer.append(line)

            if len("\n".join(buffer)) > self.chunk_size:
                chunks.append(self._create_chunk(buffer, current_section))
                buffer = []

        if buffer:
            chunks.append(self._create_chunk(buffer, current_section))
        return chunks

    def _create_chunk(self, buffer: list, section: str) -> dict:

```

```

return {
    "text": "\n".join(buffer),
    "section": section,
    "citation_key": f"{hash(''.join(buffer[:3])):x}" # Unique content-based ID
}

```

✓ Vector Database

```

class VectorDBManager:
    def __init__(self, persist_directory: str = "./chroma_db"):
        # Remove any existing persistent directory to force a fresh initialization.
        if os.path.exists(persist_directory):
            shutil.rmtree(persist_directory)
            print(f"Deleted existing database at {persist_directory}.")
        else:
            print("No existing database directory found.")

        # Create settings for the Chromadb client with the persistent directory.
        settings = Settings(persist_directory=persist_directory)
        self.client = chromadb.Client(settings=settings)

        # Initialize the embedding function.
        self.embed_model = embedding_functions.SentenceTransformerEmbeddingFunction("all-mpnet-base-v2")
        collection_name = "literature_review"

        # Use get_or_create to either create a new collection or retrieve the existing one.
        self.collection = self.client.create_collection(
            name=collection_name,
            embedding_function=self.embed_model,
            metadata={"hnsw:space": "cosine"},
            get_or_create=True
        )
        print("VectorDBManager initialized successfully with a fresh database.")

    def store_articles(self, articles: List[dict]):
        """Store articles with enhanced academic metadata."""
        ids, documents, metadatas = [], [], []
        for article in articles:
            for chunk in article["chunks"]:
                ids.append(f"{article['id']}_{{chunk['citation_key']}}")
                documents.append(chunk["text"])
                # Proper author handling
                authors = article.get("authors", "Unknown")
                if isinstance(authors, list):
                    authors_str = ", ".join(authors)
                else:
                    authors_str = str(authors)
                metadatas.append({
                    "title": article["title"],
                    "authors": authors_str,
                    "year": article["year"],
                    "section": chunk["section"],
                    "citation_key": chunk["citation_key"],
                    "source": "arxiv",
                    "pdf_url": article.get("pdf_url", "No PDF URL")
                })
            self.collection.upsert(
                documents=documents,
                metadatas=metadatas,
                ids=[f"{article['id']}_{{idx}}_{{uuid.uuid4()}}" for idx in range(len(documents))]
            )
        print("Articles stored successfully.")

```

✓ Data Extraction Module

```

class DataExtractor:
    def __init__(self, openai_client, vector_db, pdf_downloader):
        self.client = openai_client
        self.vector_db = vector_db
        self.pdf_downloader = pdf_downloader

```

```

self.chunk_size = 1500 # Optimal for academic papers

def process_articles(self, screened_articles: List[Dict]) -> List[Dict]:
    processed = []
    for article in screened_articles:
        try:
            full_text = self.pdf_downloader.get_full_text(article)
            print(f"PDF extracted for '{article['title']}' (length: {len(full_text)})")
            enriched = self._enrich_metadata(article, full_text)
            # Log before vector upsert:
            print(f"Storing {len(enriched.get('chunks', []))} chunks for article '{article['title']}' into vector DB.")
            self._store_in_vector_db(enriched)
            processed.append(enriched)
        except Exception as e:
            print(f"Failed to process {article['title']}: {str(e)}")
    return processed

def _enrich_metadata(self, article: Dict, full_text: str) -> Dict:
    """
    Enhance article data with extracted information.
    This update adds automatic extraction of the references section using Anystyle.
    """
    return {
        **article,
        "full_text": full_text,
        "chunks": self._chunk_text(full_text, article),
        "keywords": self._extract_keywords(full_text)
    }

def _chunk_text(self, text: str, article: dict) -> List[Dict]:
    """Academic-aware text segmentation"""
    return [{
        "text": text[i:i + self.chunk_size],
        "citation_key": f"{article['authors'][0]}_{article['year']}_chunk{idx}"
    } for idx, i in enumerate(range(0, len(text), self.chunk_size))]

def _extract_keywords(self, text: str) -> List[str]:
    """GPT-4 assisted keyword extraction"""
    try:
        response = self.client.chat.completions.create(
            model="gpt-4-1106-preview",
            messages=[{
                "role": "system",
                "content": "Extract 5-10 keywords from this text. Return as JSON array."
            }, {
                "role": "user",
                "content": text[:5000] # First 5k chars for cost efficiency
            }],
            response_format={"type": "json_object"}
        )
        return json.loads(response.choices[0].message.content).get("keywords", [])
    except Exception as e:
        print(f"Keyword extraction failed: {str(e)}")
        return []

def _store_in_vector_db(self, article: Dict):
    """Vector storage with academic metadata"""
    documents = []
    metadatas = []

    # PROCESS AUTHORS PROPERLY
    authors = article.get("authors", "Unknown")
    if isinstance(authors, list):
        processed_authors = ", ".join(authors)
    else:
        processed_authors = str(authors)

    for idx, chunk in enumerate(article["chunks"]):
        documents.append(chunk["text"])
        metadatas.append({
            "title": article["title"],
            "authors": processed_authors, # Consistent string format
            "year": article["year"],
            "citation": chunk["citation_key"],
            "source": "arxiv"
        })

```

```

self.vector_db.collection.add(
    documents=documents,
    metadatas=metadatas,
    ids=[f"{article['id']}_chunk{idx}" for idx in range(len(documents))]
)

```

✂ Methodology Frequency and Explanation

```

def preprocess_text(text: str) -> str:
    """
    Pre-process the text to remove irrelevant parts.
    Filters out lines containing keywords such as "references", "bibliography",
    "acknowledgement", or "conclusion" (case-insensitive).
    """
    filtered_lines = []
    for line in text.splitlines():
        lower_line = line.lower()
        if any(keyword in lower_line for keyword in ["references", "bibliography", "acknowledgement", "conclusion"]):
            continue
        filtered_lines.append(line)
    return "\n".join(filtered_lines)

# Synchronous version of methodology extraction.
def extract_methodologies_sync(article: dict, model="gpt-3.5-turbo") -> list:
    """
    Synchronously extracts research methodologies from an article's full text.
    Uses OpenAI's synchronous API call.
    """
    # Use the full text of the article and preprocess it.
    methods_text = article.get("full_text", "")
    methods_text = preprocess_text(methods_text)

    # Truncate text to avoid exceeding token limits.
    max_chars = 4000
    if len(methods_text) > max_chars:
        methods_text = methods_text[:max_chars]
    if not methods_text.strip():
        return []

    # Construct prompt to extract methodologies.
    prompt = (
        "Identify and extract the key *academic research methodologies* described in the text below. "
        "Examples of valid methodologies might include experimental design, case studies, interviews, "
        "surveys, grounded theory, data analysis techniques, etc. "
        "Exclude domain-specific topics that are not actually research methods. "
        "Return your answer as a comma-separated list, in lowercase, with no extra commentary.\n\n"
        f"Text:\n{methods_text}"
    )

    # Synchronously call the OpenAI API.
    response = openai.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    extracted = response.choices[0].message.content.strip()
    methodologies = [m.strip().lower() for m in extracted.split(",") if m.strip()]
    return methodologies

# Synchronous version of the analysis (concurrent extraction across articles).
def analyze_methodologies_sync(articles: list, max_workers: int = 5) -> dict:
    """
    Synchronously extracts methodologies from a list of articles concurrently using a thread pool,
    then aggregates the results into a frequency dictionary.
    """
    executor = ThreadPoolExecutor(max_workers=max_workers)
    futures = {executor.submit(extract_methodologies_sync, article): article for article in articles}
    all_methods = []
    for future in as_completed(futures):
        try:
            methods = future.result()
            all_methods.extend(methods)

```

```

except Exception as e:
    logging.error(f"Error in methodology extraction: {e}")
executor.shutdown(wait=True)
frequency = Counter(all_methods)
return dict(frequency)

# Synchronous version of explaining a methodology.
def explain_methodology_sync(method: str, research_objective: str, client) -> str:
    """
    Synchronously generates a detailed explanation for a research methodology.
    """
    prompt = (
        f"Explain the research methodology '{method}' in detail. "
        "Provide a step-by-step outline of how to conduct a study using this method, mention any challenges "
        "or critical considerations, and discuss its applicability to a research objective such as: "
        f"'{research_objective}'."
    )
    response = client.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    explanation = response.choices[0].message.content.strip()
    return explanation

# Synchronous version of obtaining methodology insights.
def get_methodology_insights_sync(frequency: dict, research_objective: str, client) -> dict:
    """
    Synchronously obtains detailed explanations for each methodology and bundles them with their frequencies.
    """
    insights = {}
    for method, freq in frequency.items():
        explanation = explain_methodology_sync(method, research_objective, client)
        insights[method] = {"frequency": freq, "explanation": explanation}
    return insights

def plot_top_five_methodologies(frequency: dict):
    """
    Creates a bar chart using Plotly Express that shows only the top five research methodologies.
    """
    # Sort the frequency dictionary in descending order and take the top three items.
    sorted_methods = sorted(frequency.items(), key=lambda item: item[1], reverse=True)[:5]
    top_three = dict(sorted_methods)

    # Create a DataFrame from the top five methods.
    df = pd.DataFrame(list(top_three.items()), columns=["Methodology", "Count"])
    # Plot the bar chart.
    fig = px.bar(
        df,
        x="Methodology",
        y="Count",
        title="Top 5 Research Methodologies",
        labels={"Methodology": "Research Methodology", "Count": "Frequency"}
    )
    fig.update_layout(transition_duration=500)
    return fig

```

✓ Quality Assessment and PRISMA Integration Module

```

class PrismaSummary:
    def __init__(self, quality_threshold: float = 0.5):
        """
        Initialize with a quality threshold.
        The quality score is based solely on the semantic similarity score.
        Only articles with a semantic similarity score of at least this threshold will be included.
        """
        self.quality_threshold = quality_threshold
        self.current_year = datetime.datetime.now().year

    def assess_quality(self, papers: List[Dict]) -> List[Dict]:
        """
        Assess the quality of each paper using the semantic similarity score.
        Only papers with a semantic similarity score of at least the threshold are included.

```

```

Only papers with a semantic similarity score of at least the threshold are included.
Assumes each paper contains a key "semantic_similarity_score".
"""
quality_assessed = []
for paper in papers:
    similarity_score = float(paper.get("semantic_similarity_score", 0))
    paper["quality_score"] = similarity_score
    if similarity_score >= self.quality_threshold:
        quality_assessed.append(paper)
return quality_assessed

def generate_text_summary(
    self,
    retrieved: int,
    screened: int,
    duplicates_removed: int,
    quality_assessed: int,
    included: int,
    screening_details: Dict[str, int],
    qa_excluded: int
) -> str:
    """
    Generate a detailed text summary of the PRISMA flow including granular exclusion details.
    Now includes QA exclusions as well.
    """
    # Sum all screening exclusions
    screening_exclusions = sum(screening_details.values())
    # Combine screening + QA for total
    total_excluded = screening_exclusions + qa_excluded

    summary = "PRISMA Flow Diagram Summary:\n\n"
    summary += f"Records identified from retrieval: {retrieved}\n"
    summary += f"Records after screening: {screened}\n"
    summary += f"Duplicates removed: {duplicates_removed}\n"
    summary += f"Records assessed for quality: {quality_assessed}\n"
    summary += f"Records included in the review: {included}\n"
    summary += f"Total records excluded: {total_excluded}\n"

    summary += "Reasons for exclusion:\n"
    for reason, count in screening_details.items():
        summary += f" - {reason}: {count}\n"
    summary += f" - Quality Assessment: {qa_excluded}\n"

    return summary

def create_interactive_visualization(
    self,
    retrieved: int,
    screened: int,
    duplicates_removed: int,
    quality_assessed: int,
    included: int,
    screening_details: Dict[str, int]
) -> go.Figure:
    """
    Show each screening reason separately, plus a node for QA exclusion.
    """

    # 1. Sum screening exclusions (e.g., publication year, relevance)
    screening_exclusions = sum(screening_details.values())
    # 2. Compute QA exclusions
    qa_exclusions = screened - duplicates_removed - quality_assessed

    # --- Build dynamic labels ---
    labels = ["Retrieved"] # Node 0
    # Next: one node per screening reason
    reason_nodes = [f"Excluded: {r}" for r in screening_details.keys()]
    labels.extend(reason_nodes)
    # Then: "Screened", "Excluded: Quality Assessment", "Quality Assessed", "Included"
    labels.extend(["Screened", "Excluded: Quality Assessment", "Quality Assessed", "Included"])

    # Indices
    retrieved_idx = 0
    reason_start_idx = 1
    # The index after all reason nodes
    screened_idx = reason_start_idx + len(screening_details)
    qa_excluded_idx = screened_idx + 1

```

```

qa_passed_idx = screened_idx + 2
included_idx = screened_idx + 3

# Build flows
sources, targets, values = [], [], []

# 1. Retrieved → each screening reason
i = 0
for reason, count in screening_details.items():
    sources.append(retrieved_idx)
    targets.append(reason_start_idx + i)
    values.append(count)
    i += 1

# 2. Retrieved → Screened
sources.append(retrieved_idx)
targets.append(screened_idx)
# Those that pass screening
pass_screening = retrieved - screening_exclusions
values.append(pass_screening)

# 3. Screened → Excluded: Quality Assessment
sources.append(screened_idx)
targets.append(qa_excluded_idx)
values.append(qa_exclusions)

# 4. Screened → Quality Assessed
sources.append(screened_idx)
targets.append(qa_passed_idx)
values.append(quality_assessed)

# 5. Quality Assessed → Included
sources.append(qa_passed_idx)
targets.append(included_idx)
values.append(included)

fig = go.Figure(data=[go.Sankey(
    node=dict(
        pad=15,
        thickness=20,
        line=dict(color="black", width=0.5),
        label=labels,
    ),
    link=dict(
        source=sources,
        target=targets,
        value=values
    )
)])

fig.update_layout(title_text="PRISMA Flow Diagram", font_size=10)
return fig

def export_prisma_summary(self, summary: str, filename: str = "prisma_report.txt") -> None:
    """
    Export the PRISMA summary to a text file.
    """
    with open(filename, "w", encoding="utf-8") as f:
        f.write(summary)
    print(f"Exported PRISMA summary to {filename}")

def export_to_rayyan_csv(self, papers: List[Dict], filename: str = "rayyan_import.csv") -> None:
    """
    Export the list of papers to a CSV file formatted for Rayyan import.
    Logs a warning if an article has an empty "abstract" field.
    """
    fieldnames = ["Title", "Abstract", "Year", "Link", "Source"]
    with open(filename, mode="w", encoding="utf-8", newline="") as csv_file:
        writer = csv.DictWriter(csv_file, fieldnames=fieldnames)
        writer.writeheader()
        for paper in papers:
            abstract = paper.get("abstract", "").strip()
            # Assert that abstract is not empty; log a warning and use a default if it is.
            if not abstract:
                logging.warning(f"Paper titled '{paper.get('title')}' has an empty abstract. Setting to default value.")
                abstract = "No Abstract Available"
            writer.writerow({
                "Title": paper.get("title", ""),
                "Abstract": abstract,
                "Year": paper.get("year", ""),
                "Link": paper.get("link", ""),
                "Source": paper.get("source", "")
            })

```

```

        "title": paper.get("title", "No Title"),
        "Abstract": abstract,
        "Year": paper.get("year", "Unknown"),
        "Link": paper.get("link", "No Link Available"),
        "Source": paper.get("source", "Unknown")
    })
    logging.info(f"Exported {len(papers)} articles to {filename}")

```

✓ Visualization Module

```

class VisualizationModule:
    def __init__(self, papers: list):
        """
        Initialize with a list of article dictionaries.
        Each article should have keys: "year", "abstract", "title", "source", "link",
        and "semantic_similarity_score" (already computed).
        """
        self.papers = papers

    def create_scatter_plot(self, research_objective: str):
        """
        Create an interactive scatter plot with Publication Year on the x-axis and
        Semantic Similarity Score on the y-axis.
        Assumes each article already has a key 'semantic_similarity_score'.
        """
        # Construct a new list of dictionaries that includes all necessary fields.
        data = []
        for paper in self.papers:
            try:
                year = int(paper.get("year", "0"))
            except Exception:
                year = None
            similarity = paper.get("semantic_similarity_score", None)
            if similarity is None:
                raise ValueError("One or more articles are missing the 'semantic_similarity_score' key.")
            data.append({
                "Year": year,
                "Title": paper.get("title", "No Title"),
                "Source": paper.get("source", "Unknown"),
                "Link": paper.get("link", "No Link"),
                "Similarity": similarity
            })

        # Create a DataFrame from the collected data.
        df_new = pd.DataFrame(data)

        # Create the scatter plot using Plotly Express.
        fig = px.scatter(
            df_new,
            x="Year",
            y="Similarity",
            hover_data=["Title", "Source", "Link"],
            title="Publication Year vs. Semantic Similarity Score",
            labels={"Year": "Publication Year", "Similarity": "Semantic Similarity Score"}
        )
        fig.update_layout(transition_duration=500)
        return fig

    def create_bar_chart_by_year(self):
        """
        Create a bar chart showing the number of articles per publication year.
        """
        data = []
        for paper in self.papers:
            try:
                year = int(paper.get("year", "0"))
            except Exception:
                year = None
            data.append({
                "Year": year,
                "Title": paper.get("title", "No Title"),
                "Source": paper.get("source", "Unknown"),

```



```

        "Link": paper.get("link", "No Link")
    })
df = pd.DataFrame(data)
df_year = df.groupby("Year").size().reset_index(name="Count")
fig = px.bar(
    df_year,
    x="Year",
    y="Count",
    title="Number of Articles per Publication Year",
    labels={"Year": "Publication Year", "Count": "Number of Articles"}
)
fig.update_layout(transition_duration=500)
return fig

```

✓ Analysis and Synthesis Module

```

def split_text_by_tokens(text: str, max_tokens: int) -> list:
    """
    Split the input text into chunks, each with at most max_tokens tokens,
    using the tiktoken encoding for GPT-3.5-turbo.
    """
    # Use the encoding for GPT-3.5-turbo; "cl100k_base" is recommended.
    encoding = tiktoken.get_encoding("cl100k_base")
    token_ids = encoding.encode(text)
    chunks = []
    for i in range(0, len(token_ids), max_tokens):
        chunk_ids = token_ids[i:i+max_tokens]
        chunk_text = encoding.decode(chunk_ids)
        chunks.append(chunk_text)
    return chunks

def analysis_synthesis_tool(input_text: str, max_tokens: int = 1500) -> dict:
    """
    Analyzes and synthesizes the input text by splitting it into tokenized chunks
    to avoid exceeding the model's context length.

    Parameters:
        input_text (str): The full text to be analyzed.
        max_tokens (int): The maximum number of tokens per chunk.

    Returns:
        dict: A dictionary with the synthesized narrative under 'structured_response'.
    """
    llm = ChatOpenAI(model_name="gpt-3.5-turbo", temperature=0)

    # Split the input text into token-accurate chunks.
    chunks = split_text_by_tokens(input_text, max_tokens)

    synthesized_chunks = []
    prompt_template = (
        """You are an expert academic researcher. You have been provided with extracted data, including summaries, methods, results, conclus
{chunk}

Please analyze and synthesize this data into a clear academic narrative that addresses the research question. Your narrative should include:
1. A summary of the main findings and trends.
2. Identification of key research themes.
3. Discussion of the framework and/or research methods employed over time in this research area.
4. A concluding synthesis of implication of findings.
5. Inline citations (e.g., Author, Year) throughout the narrative to trace factual sources.

Ensure your response is well-structured and coherent, with logical flow and clarity."""
    )

    for chunk in chunks:
        prompt = prompt_template.format(chunk=chunk)
        response = llm.invoke([{"role": "user", "content": prompt}])
        synthesized_chunks.append(response.content.strip())

    # Combine all chunk outputs into one synthesized narrative.
    synthesized = " ".join(synthesized_chunks)
    return {"structured_response": synthesized}

```

✓ Interpretation module

```
def interpretation_tool_sync(input_text: str, model="gpt-3.5-turbo") -> str:
    """
    Synchronously interprets a synthesized narrative.
    """
    prompt = f"""
You are an expert academic researcher. Below is a synthesized narrative from a systematic literature review:

{input_text}

Please provide a detailed interpretation of the narrative. Address the following:
1. Explain the significance of the findings.
2. Discuss how these results contribute to achieving the research objective.
3. Identify any gaps, limitations, or areas needing further clarification.
4. Suggest future research directions based on the findings.
5. Provide inline citations (e.g., Author, Year) to support your interpretation.

Your response should be a clear, structured interpretation with distinct sections for each point.
"""
    response = openai.chat.completions.create(
        model=model,
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    interpreted = response.choices[0].message.content.strip()
    return interpreted

def run_interpretation_sync(synthesized_text: str) -> str:
    """
    Runs the interpretation synchronously.
    """
    return interpretation_tool_sync(synthesized_text)
```

✓ Literature Review Writing Module

```
def count_tokens(text, model="gpt-4"):
    encoding = tiktoken.encoding_for_model(model)
    return len(encoding.encode(text))

def literature_review_writing_tool_sync(inputs: dict, methodology_insights: str) -> str:
    """
    Synchronously writes the literature review draft using chunking and error handling
    to avoid exceeding token limits.
    """
    # Construct the full prompt
    base_prompt = f"""You are an expert academic writer. Synthesize a concise and comprehensive literature review draft using the following
- **Synthesized Narrative**: {inputs['synthesized_narrative']}
- **Interpreted Summary**: {inputs['interpreted_summary']}
- **PRISMA Summary**: {inputs['prisma_summary']}
- **Methodology Insights**: {methodology_insights}

Ensure the review is well-structured, clearly written, and appropriately integrates these components into a cohesive narrative.
"""

    # Maximum tokens available for the prompt (reserve some for the completion)
    max_context_tokens = 8192 - 500 # adjust reserve as needed

    # If the prompt is too long, break it into smaller chunks.
    if count_tokens(base_prompt, model="gpt-4") > max_context_tokens:
        # Define an initial chunk size in characters (heuristic)
        chunk_size = 10000
        chunks = []
        for i in range(0, len(base_prompt), chunk_size):
            chunk = base_prompt[i:i + chunk_size]
            # Ensure the chunk is within the token limit by trimming if needed.
            while count_tokens(chunk, model="gpt-4") > max_context_tokens and len(chunk) > 0:
                chunk = chunk[:-100] # trim off 100 characters and check again
            chunks.append(chunk)
        else:
```

```

chunks = [base_prompt]

response_text = ""
for chunk in chunks:
    try:
        response = openai.chat.completions.create(
            model="gpt-4",
            messages=[{"role": "user", "content": chunk}],
            temperature=0.7,
            max_tokens=500 # reduce max_tokens to keep total within limits
        )
        response_text += response.choices[0].message.content.strip()
        # Short delay between chunks
        time.sleep(1)
    except RateLimitError as e:
        print(f"Rate limit error encountered: {e}. Waiting...")
        time.sleep(10) # wait longer before retrying
    except BadRequestError as e:
        print(f"Invalid request error encountered: {e}. Likely due to exceeding context length. Consider further reducing input size.")
    except Exception as e:
        print(f"An unexpected error occurred: {e}")

return response_text

def run_literature_review_writing_sync(inputs: dict, methodology_insights: str) -> str:
    """
    Synchronously runs the literature review writing tool.
    """
    return literature_review_writing_tool_sync(inputs, methodology_insights)

```

✓ Ethics, Transparency, and Documentation Module

```

def generate_ethics_documentation(tool_logs: dict, decision_logs: dict, ethical_considerations: str) -> str:
    """
    Combines AI tool logs, decision-making logs, and ethical considerations into a final documentation report.
    Writes the report to a file and returns the documentation string.

    Parameters:
        tool_logs (dict): Logs detailing the usage and outputs of each AI tool.
        decision_logs (dict): Logs describing the decision-making process during the review.
        ethical_considerations (str): A summary of ethical considerations (e.g., bias prevention, plagiarism checks).

    Returns:
        str: A formatted documentation report.
    """
    timestamp = datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S")
    documentation = f"Ethics, Transparency, and Documentation Report\n"
    documentation += f"Generated on: {timestamp}\n\n"
    documentation += "Tool Usage Logs:\n"
    documentation += json.dumps(tool_logs, indent=2) + "\n\n"
    documentation += "Decision-Making Process Logs:\n"
    documentation += json.dumps(decision_logs, indent=2) + "\n\n"
    documentation += "Ethical Considerations:\n"
    documentation += ethical_considerations + "\n\n"

    # Write the documentation to a file
    with open("ethics_documentation.txt", "w", encoding="utf-8") as f:
        f.write(documentation)

    return documentation

```

✓ LLM Judge Module

```
def llm_judge_sync(text: str) -> str:
    """
    Synchronously uses the OpenAI API to critique the provided output from the literature review pipeline.
    It evaluates the text on factual accuracy, logical coherence, relevance, and completeness,
    and returns detailed feedback.
    """
    prompt = f"""
You are an expert academic reviewer. Your task is to evaluate the following text output from a systematic literature review pipeline. Analyz
1. Factual Accuracy: Are the key facts correct and well-supported?
2. Logical Coherence: Is the text logically structured and easy to follow?
3. Relevance: Does the text address the research question and objectives effectively?
4. Completeness: Are there any gaps or missing details that should be included?
5. Overall Quality: Provide feedback on the text's quality and suggest areas for improvement.

Text:
{text}

Provide your feedback in a clear, structured format.
"""
    response = openai.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        temperature=0
    )
    return response.choices[0].message.content.strip()

def log_judge_feedback_sync(text: str, feedback: str, log_file: str = "llm_judge_feedback.log"):
    """
    Synchronously logs the output text and the judge's feedback to a log file.
    """
    with open(log_file, "a", encoding="utf-8") as f:
        f.write("=== LLM Judge Feedback ===\n")
        f.write("Reviewed Text:\n")
        f.write(text + "\n\n")
        f.write("Feedback:\n")
        f.write(feedback + "\n")
        f.write("=" * 40 + "\n\n")
```

✓ Relevance Evaluator

```
# relevance_evaluator.py
class RelevanceEvaluator:
    def __init__(self, model_name="all-mpnet-base-v2"):
        # Initialize the sentence transformer model
        self.model = SentenceTransformer(model_name)

    def evaluate(self, generated_output: str, query: str) -> float:
        """
        Evaluates the relevance of the generated output to the query using cosine similarity.

        Parameters:
            generated_output (str): The output produced by the RAG system.
            query (str): The user query or research objective.

        Returns:
            float: A similarity score between 0 and 1 representing relevance.
        """
        # Encode the query and the generated output into embeddings
        query_embedding = self.model.encode(query, convert_to_tensor=True)
        output_embedding = self.model.encode(generated_output, convert_to_tensor=True)

        # Calculate cosine similarity between the embeddings
        similarity = util.pytorch_cos_sim(query_embedding, output_embedding).item()
        return similarity
```

✓ Groundedness Evaluator

```
# groundedness_evaluator.py

class GroundednessEvaluator:
    def __init__(self, model_name="all-mpnet-base-v2"):
        # Initialize the sentence transformer model
        self.model = SentenceTransformer(model_name)

    def evaluate(self, generated_output: str, retrieved_docs: list) -> float:
        """
        Evaluates the groundedness of the generated output based on the retrieved documents.

        Parameters:
            generated_output (str): The output produced by the RAG system.
            retrieved_docs (list of str): A list of documents (or excerpts) retrieved from the source.

        Returns:
            float: A groundedness score between 0 and 1.
        """
        if not retrieved_docs:
            # If there are no retrieved documents, we consider the output to have no grounding.
            return 0.0

        # Encode the generated output
        output_embedding = self.model.encode(generated_output, convert_to_tensor=True)

        # Compute similarity scores for each retrieved document
        similarities = []
        for doc in retrieved_docs:
            doc_embedding = self.model.encode(doc, convert_to_tensor=True)
            similarity = util.pytorch_cos_sim(output_embedding, doc_embedding).item()
            similarities.append(similarity)

        # Aggregate similarities. Here we use the average as a proxy measure.
        groundedness_score = sum(similarities) / len(similarities)
        return groundedness_score
```

✓ Retrieval Relevance Evaluator

```
# retrieval_relevance_evaluator.py
class RetrievalRelevanceEvaluator:
    def __init__(self, model_name="all-mpnet-base-v2"):
        # Initialize the sentence transformer model
        self.model = SentenceTransformer(model_name)

    def evaluate(self, query: str, retrieved_docs: list) -> float:
        """
        Evaluates the relevance of the retrieved documents relative to the query.

        Parameters:
            query (str): The input query or research objective.
            retrieved_docs (list of str): A list of documents (or excerpts) retrieved by the system.

        Returns:
            float: A retrieval relevance score between 0 and 1.
        """
        if not retrieved_docs:
            # If no documents were retrieved, return a score of 0
            return 0.0

        # Encode the query
        query_embedding = self.model.encode(query, convert_to_tensor=True)

        # Compute cosine similarity between the query and each retrieved document
        similarities = []
        for doc in retrieved_docs:
            doc_embedding = self.model.encode(doc, convert_to_tensor=True)
            similarity = util.pytorch_cos_sim(query_embedding, doc_embedding).item()
            similarities.append(similarity)

        # Aggregate the similarities; here we take the average as the overall relevance score.
        relevance_score = sum(similarities) / len(similarities)
        return relevance_score
```

✓ Evaluation Driver

Evaluation Driver Code

```
class EvaluationDriver:
    def __init__(self):
        self.relevance_evaluator = RelevanceEvaluator()
        self.groundedness_evaluator = GroundednessEvaluator()
        self.retrieval_relevance_evaluator = RetrievalRelevanceEvaluator()

    def evaluate(self, query: str, generated_output: str, retrieved_docs: list) -> dict:
        relevance_score = self.relevance_evaluator.evaluate(generated_output, query)
        groundedness_score = self.groundedness_evaluator.evaluate(generated_output, retrieved_docs)
        retrieval_relevance_score = self.retrieval_relevance_evaluator.evaluate(query, retrieved_docs)

        return {
            "relevance_score": relevance_score,
            "groundedness_score": groundedness_score,
            "retrieval_relevance_score": retrieval_relevance_score
        }
```

✓ Aggregation and Logging of Evaluation Results

```
def log_evaluation_results(query: str, final_review: str, evaluation_results: dict, log_file: str = "evaluation_log.csv"):
    """
    Logs the evaluation results along with the query and a brief excerpt from the final review into a CSV file.

    Parameters:
    - query: The input query used for the pipeline run.
    - final_review: The generated final literature review.
    - evaluation_results: A dictionary containing the scores from the evaluation driver.
    - log_file: The CSV file to which results are appended.
    """
    # Define the fields to log.
    fieldnames = [
        "timestamp",
        "query",
        "relevance_score",
        "groundedness_score",
        "retrieval_relevance_score",
        "final_review_excerpt"
    ]

    timestamp = datetime.datetime.now().isoformat()
    # Extract a brief excerpt from the final review (first 100 characters).
    review_excerpt = final_review[:100]

    # Check if the log file already exists.
    file_exists = os.path.exists(log_file)

    # Open the log file in append mode.
    with open(log_file, "a", newline="", encoding="utf-8") as csvfile:
        writer = csv.DictWriter(csvfile, fieldnames=fieldnames)
        # If the file is new, write the header.
        if not file_exists:
            writer.writeheader()
        # Write the evaluation record.
        writer.writerow({
            "timestamp": timestamp,
            "query": query,
            "relevance_score": evaluation_results.get("relevance_score", 0),
            "groundedness_score": evaluation_results.get("groundedness_score", 0),
            "retrieval_relevance_score": evaluation_results.get("retrieval_relevance_score", 0),
            "final_review_excerpt": review_excerpt
        })
    print(f"Evaluation results logged to {log_file}")

def aggregate_evaluation_logs(log_file: str = "evaluation_log.csv"):
    """
    Aggregates evaluation scores from the CSV log file and prints the average scores.
```

```

Parameters:
- log_file: The CSV file containing logged evaluation records.
"""
if not os.path.exists(log_file):
    print("No evaluation logs found.")
    return

relevance_scores = []
groundedness_scores = []
retrieval_relevance_scores = []

with open(log_file, "r", newline="", encoding="utf-8") as csvfile:
    reader = csv.DictReader(csvfile)
    for row in reader:
        try:
            relevance_scores.append(float(row["relevance_score"]))
            groundedness_scores.append(float(row["groundedness_score"]))
            retrieval_relevance_scores.append(float(row["retrieval_relevance_score"]))
        except Exception as e:
            print("Error parsing row:", row, e)

avg_relevance = sum(relevance_scores) / len(relevance_scores) if relevance_scores else 0
avg_groundness = sum(groundedness_scores) / len(groundedness_scores) if groundedness_scores else 0
avg_retrieval_relevance = sum(retrieval_relevance_scores) / len(retrieval_relevance_scores) if retrieval_relevance_scores else 0

print("\n=== Aggregated Evaluation Metrics ===")
print(f"Average Relevance Score: {avg_relevance:.2f}")
print(f"Average Groundedness Score: {avg_groundness:.2f}")
print(f"Average Retrieval Relevance Score: {avg_retrieval_relevance:.2f}")

# -----
# Integration Example:
# -----
# At the end of your end-to-end pipeline (after obtaining final_review and evaluation_results),
# call the log_evaluation_results function:

# Assuming you have these variables from your pipeline:
# - query: the original query string.
# - final_review: the final literature review draft generated.
# - evaluation_results: the dictionary output from EvaluationDriver.evaluate(...)

# Example:
# log_evaluation_results(query, final_review, evaluation_results)

# You can later run the aggregation function to see the overall metrics:
# aggregate_evaluation_logs()

```

✓ Pipeline

```

def main_pipeline_end_to_end_sync(query: str, research_objective: str, start_year: int, min_citations: int, limit: int = 30) -> dict:
    logging.info("=== Starting End-to-End Pipeline ===")

    # 1. Review Question Formulation
    rqf = ReviewQuestionFormulator()
    keywords_list = rqf.extract_keywords(research_objective)
    print("Extracted Keywords from Research Objective:")
    print(keywords_list)

    review_questions_with_pcc = rqf.generate_review_questions_with_pcc(query)
    print("\nGenerated Review Questions with PCC Elements:")
    print(json.dumps(review_questions_with_pcc, indent=2))

    # Create an instance of the TIM Pathway Mapper
    tim_mapper = TIMPathwayMapper()

    # For each generated review question, map the PCC elements to a pathway and print the result
    for idx, question in enumerate(review_questions_with_pcc):
        mapped_pathway = tim_mapper.map_to_pathway(question["pcc"])
        print(f"\nQuestion {idx+1}: {question['question']}")
        print("PCC Elements:", question["pcc"])
        print("Mapped Pathway:", mapped_pathway)

```

```

# 2. Paper Retrieval
pcc_elements = {"population": "", "concept": "", "context": ""}
tim_mapper = TIMPathwayMapper()
retriever = PaperRetriever(api_key="", tim_mapper=tim_mapper)
questions_with_pcc = [{"question": query, "pcc": pcc_elements}]
articles = retriever.unified_retrieval(questions_with_pcc, max_results=limit)
print("\nNumber of articles retrieved:", len(articles))
if articles:
    print("Sample Retrieved Article:")
    print(json.dumps(articles[0], indent=2))

# 3. Screening and Duplicate Removal
screening = ArticleScreening()
screened_articles, exclusion_details = screening.screen_articles(articles, start_year=start_year, relevance_keywords=keywords_list)
print("\nNumber of articles after screening:", len(screened_articles))

deduped_articles = screening.remove_duplicates(screened_articles)
duplicates_removed = len(screened_articles) - len(deduped_articles)
print("Number of articles after deduplication:", len(deduped_articles))

# 4. Compute Semantic Similarity Scores
semantic_sim = SemanticSimilarity(model_name='all-mpnet-base-v2')
deduped_articles = semantic_sim.add_similarity_scores(deduped_articles, research_objective)
print("\nSemantic similarity scores computed for deduplicated articles.")

# 5. Quality Assessment and PRISMA Summary Generation
prisma_summary = PrismaSummary(quality_threshold=0.5)
quality_assessed_articles = prisma_summary.assess_quality(deduped_articles)
qa_excluded = len(deduped_articles) - len(quality_assessed_articles)
quality_assessed_count = len(quality_assessed_articles)
included = quality_assessed_count
print("\nNumber of articles after quality assessment:", quality_assessed_count)
print("Number of articles excluded during QA:", qa_excluded)

retrieved_count = len(articles)
screened_count = len(screened_articles)
prisma_text_summary = prisma_summary.generate_text_summary(
    retrieved=retrieved_count,
    screened=screened_count,
    duplicates_removed=duplicates_removed,
    quality_assessed=quality_assessed_count,
    included=included,
    screening_details=exclusion_details,
    qa_excluded=qa_excluded
)
print("\n--- PRISMA Text Summary ---\n")
print(prisma_text_summary)

sankey_fig = prisma_summary.create_interactive_visualization(
    retrieved=retrieved_count,
    screened=screened_count,
    duplicates_removed=duplicates_removed,
    quality_assessed=quality_assessed_count,
    included=included,
    screening_details=exclusion_details
)

# Export PRISMA outputs
with open("prisma_report.txt", "w") as f:
    f.write(prisma_text_summary)
sankey_fig.write_html("prisma_sankey.html")
prisma_summary.export_to_rayyan_csv(quality_assessed_articles)

# 7. Data Extraction and Vector DB Storage
vector_db = VectorDBManager(persist_directory="./chroma_db")
pdf_downloader = PDFDownloader()
data_extractor = DataExtractor(openai_client=openai, vector_db=vector_db, pdf_downloader=pdf_downloader)
extracted_data = data_extractor.process_articles(deduped_articles)
print("\n=== Data Extraction Test ===")
print("Number of articles processed:", len(extracted_data))
if extracted_data:
    sample_article = extracted_data[0]
    print("Sample Enriched Article:")
    print("Title:", sample_article.get("title"))
    print("Extracted Keywords:", sample_article.get("keywords"))
    print("Number of chunks:", len(sample_article.get("chunks", [])))

# 8. Methodology Extraction and Analysis (Synchronous)

```



```

methodology_frequency = analyze_methodologies_sync(extracted_data, max_workers=5)
print("\n=== Methodology Extraction Test ===")
print("Methodology frequency:", methodology_frequency)
methodology_insights = get_methodology_insights_sync(methodology_frequency, research_objective, client)
print("\nMethodology Insights:")
print(json.dumps(methodology_insights, indent=2))
# Generate the Plotly figure
methodology_fig = plot_top_five_methodologies(methodology_frequency)

# 9. Analysis and Synthesis
analysis_result = analysis_synthesis_tool(json.dumps(extracted_data, indent=2))
synthesized_narrative = analysis_result.get("structured_response", "No output produced.")
print("\n=== Analysis and Synthesis Test ===")
print("Synthesized Narrative:\n", synthesized_narrative)

# 10. Interpretation
interpreted_summary = run_interpretation_sync(synthesized_narrative)
print("\n=== Interpretation Test ===")
print("Interpreted Summary:\n", interpreted_summary)

# 11. Literature Review Writing
inputs = {
    "synthesized_narrative": synthesized_narrative,
    "interpreted_summary": interpreted_summary,
    "prisma_summary": prisma_text_summary
}
final_review = run_literature_review_writing_sync(inputs, json.dumps(methodology_insights, indent=2))
print("\n=== Literature Review Writing Test ===")
print("Final Literature Review Draft:\n", final_review)

# --- Step 12: Evaluation of RAG Output ---

# Use the final literature review draft as the generated output.
# For retrieved documents, we extract (for example) the abstracts from the quality-assessed articles.
retrieved_docs_texts = [article.get("abstract", "") for article in quality_assessed_articles]

# Initialize the EvaluationDriver (assumes the evaluators have been defined in previous cells).
eval_driver = EvaluationDriver()

# Evaluate the final review against the original query using the actual retrieved documents.
evaluation_results = eval_driver.evaluate(query, final_review, retrieved_docs_texts)

print("\n=== Evaluation Driver Results ===")
print(f"Relevance Score: {evaluation_results['relevance_score']:.2f}")
print(f"Groundedness Score: {evaluation_results['groundedness_score']:.2f}")
print(f"Retrieval Relevance Score: {evaluation_results['retrieval_relevance_score']:.2f}")

# Log the current run's evaluation metrics to the CSV file.
log_evaluation_results(query, final_review, evaluation_results)

# 13. LLM Judge for Quality Control
judge_feedback = llm_judge_sync(final_review)
print("LLM Judge Feedback:\n", judge_feedback)
log_judge_feedback_sync(final_review, judge_feedback)

# 14. Additional Visualizations from VisualizationModule
vis_module = VisualizationModule(deduped_articles)
if not vis_module.papers:
    print("No articles available for additional visualization.")
    bar_chart = px.bar(title="No Data to Display")
    scatter_chart = px.scatter(title="No Data to Display")
else:
    bar_chart = vis_module.create_bar_chart_by_year()
    scatter_chart = vis_module.create_scatter_plot(research_objective)
    bar_chart.show()
    scatter_chart.show()

# --- Append Evaluation Results to Pipeline Output ---
results = {
    "final_articles": quality_assessed_articles,
    "prisma_text_summary": prisma_text_summary,
    "sankey_diagram": sankey_fig,
    "data_extraction": extracted_data,
    "methodology_insights": methodology_insights,
    "synthesized_narrative": synthesized_narrative,
    "interpreted_summary": interpreted_summary,
    "final_review": final_review,

```

```

    "llm_judge_feedback": judge_feedback,
    "evaluation_results": evaluation_results,
    "methodology_fig": methodology_fig,
    "bar_chart": bar_chart,
    "scatter_chart": scatter_chart
}

logging.info("=== End-to-End Pipeline Execution Complete ===")
return results

```

✓ Gradio UI

```

def run_pipeline_wrapper(query, research_objective, start_year, limit):
    try:
        results = main_pipeline_end_to_end_sync(query, research_objective, start_year, 0, limit)
        methodology_fig = results.get("methodology_fig", None)
        # Format evaluation results as a pretty JSON string (if available)
        evaluation_results = results.get("evaluation_results", {})
        evaluation_results_str = json.dumps(evaluation_results, indent=2) if evaluation_results else "No evaluation results available"

        return [
            results.get("final_review", "No review generated"),
            results.get("llm_judge_feedback", "No feedback available"),
            results.get("prisma_text_summary", "No PRISMA summary available"),
            results.get("sankey_diagram", None),
            results.get("bar_chart", None),
            results.get("scatter_chart", None),
            methodology_fig,
            evaluation_results_str
        ]
    except Exception as e:
        import traceback
        error_msg = f"Error in pipeline: {str(e)}\n{traceback.format_exc()}"
        print(error_msg)
        return [f"Error: {str(e)}", f"Error: {str(e)}", f"Error: {str(e)}", None, None, None, "", None]

def create_gradio_interface():
    with gr.Blocks(title="Systematic Literature Review") as demo:
        gr.Markdown("## Systematic Literature Review")
        gr.Markdown("This interface runs the systematic review pipeline and displays the final review, judge feedback, PRISMA summary, visualizations, and methodology.")
        with gr.Row():
            with gr.Column(scale=1):
                query_input = gr.Textbox(label="Query", placeholder="Enter your search query here")
                research_obj_input = gr.Textbox(label="Research Objective", placeholder="Enter the research objective", lines=5)
                start_year_input = gr.Number(label="Start Year", value=2020)
                limit_input = gr.Slider(label="Article Limit", minimum=10, maximum=100, step=10, value=20)
                submit_btn = gr.Button("Submit")
                clear_btn = gr.Button("Clear")
            with gr.Column(scale=2):
                final_review_output = gr.Textbox(label="Final Literature Review Draft", lines=10)
                judge_feedback_output = gr.Textbox(label="LLM Judge Feedback", lines=5)
                prisma_summary_output = gr.Textbox(label="PRISMA Summary", lines=5)
                sankey_diagram_output = gr.Plot(label="PRISMA Flow Diagram")
                year_chart_output = gr.Plot(label="Articles per Year")
                similarity_chart_output = gr.Plot(label="Publication Year vs. Semantic Similarity")
                methodology_plot_output = gr.Plot(label="Top 5 Methodologies")
                evaluation_results_output = gr.Textbox(label="Evaluation Results", lines=10)
        submit_btn.click(
            fn=run_pipeline_wrapper,
            inputs=[query_input, research_obj_input, start_year_input, limit_input],
            outputs=[
                final_review_output,
                judge_feedback_output,
                prisma_summary_output,
                sankey_diagram_output,
                year_chart_output,
                similarity_chart_output,
                methodology_plot_output,
                evaluation_results_output
            ]
        )
        clear_btn.click(
            fn=lambda: ["", "", "", None, None, None, "", None],

```

```
inputs=[],
outputs=[
    final_review_output,
    judge_feedback_output,
    prisma_summary_output,
    sankey_diagram_output,
    year_chart_output,
    similarity_chart_output,
    methodology_plot_output,
    evaluation_results_output
]
)
return demo

if __name__ == "__main__":
    demo = create_gradio_interface()
    demo.launch(debug=True, share=True)
```

 Show hidden output

✓ Evaluation Aggregation

```
aggregate_evaluation_logs()
```